



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Scalar and Predictive Quantization

Marco Cagnazzo

Multimedia Communications



Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization



Definitions

De

Ur

Ex

Ra

Op

Pr

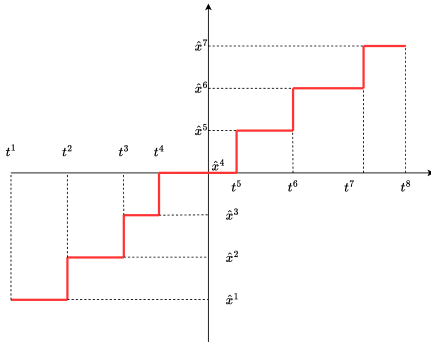


Scalar quantization (SQ): it is just a staircase function (almost always)

Here we show the graph of a possible $Q(x)$

Go to [wooclap.com](https://www.wooclap.com) with the code **PBTFUO**

Input range:





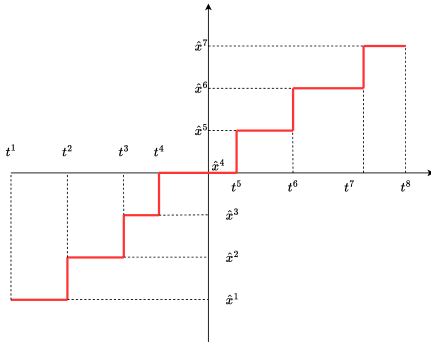
Scalar quantization (SQ): it is just a staircase function (almost always)

Here we show the graph of a possible $Q(x)$

Go to [wooclap.com](https://www.wooclap.com) with the code **PBTFUO**

Input range: (t_1, t_8)

Number of levels:





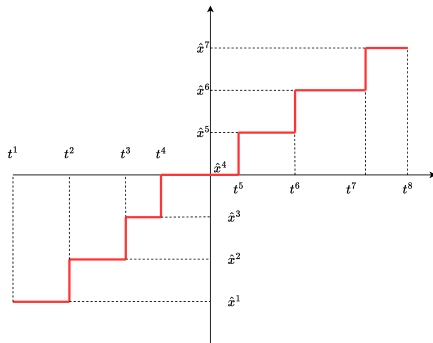
Scalar quantization (SQ): it is just a staircase function (almost always)

Here we show the graph of a possible $Q(x)$

Go to [wooclap.com](https://www.wooclap.com) with the code **PBTFUO**

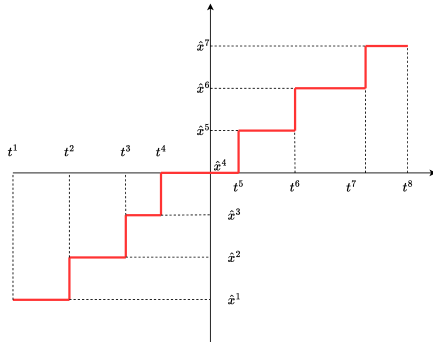
Input range: (t_1, t_8)

Number of levels: $L = 7$





Scalar quantization (SQ): it is just a staircase function (almost always)



Here we show the graph of a possible $Q(x)$

Go to [wooclap.com](https://www.wooclap.com) with the code **PBTFUO**

Input range: (t_1, t_8)

Number of levels: $L = 7$

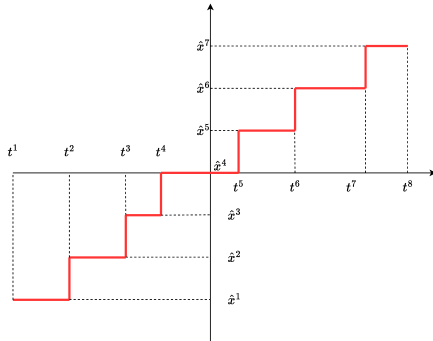
The t^i 's determine a *partition* of the input interval

Each $\Theta^i = (t^i, t^{i+1})$ is a quantization region or cell

Any number in $\Theta^i = (t^i, t^{i+1})$ is quantized as $\hat{x}^i$



Scalar quantization (SQ): it is just a staircase function (almost always)



Here we show the graph of a possible $Q(x)$

Go to [wooclap.com](https://www.wooclap.com) with the code **PBTFUO**

Input range: (t_1, t_8)

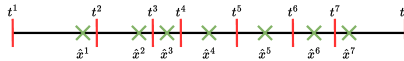
Number of levels: $L = 7$

The t^i 's determine a *partition* of the input interval

Each $\Theta^i = (t^i, t^{i+1})$ is a quantization region or cell

Any number in $\Theta^i = (t^i, t^{i+1})$ is quantized as $\hat{x}^i$

The graph below also represent the quantizer: the thresholds and the levels completely determine the quantizer





Definition : scalar quantization (SQ)

Scalar quantization (SQ) is a function Q from $\mathbb{R}$ to a discrete set called *Dictionary*

$$Q : x \in \mathbb{R} \rightarrow y \in \mathcal{C} = \{\hat{x}^1, \hat{x}^2, \dots, \hat{x}^L\} \subset \mathbb{R}$$

- ▶ $\mathcal{C}$: Dictionary, it is a discrete subset of $\mathbb{R}$
- ▶ $\hat{x}^i$: quantization level, code-word
- ▶ $e = x - Q(x)$: Quantization noise
- ▶ $\Theta^i = \{x : Q(x) = \hat{x}^i\}$: Decision regions or cells

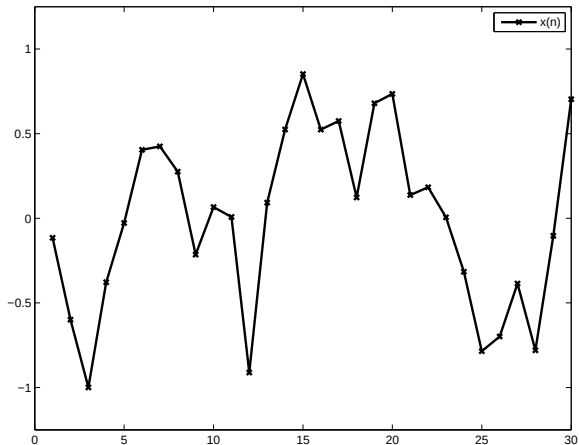
Regions and levels completely define the SQ

Regions are typically defined as intervals: $\Theta^i = (t^i, t^{i+1})$

Therefore, thresholds and levels also completely define a SQ

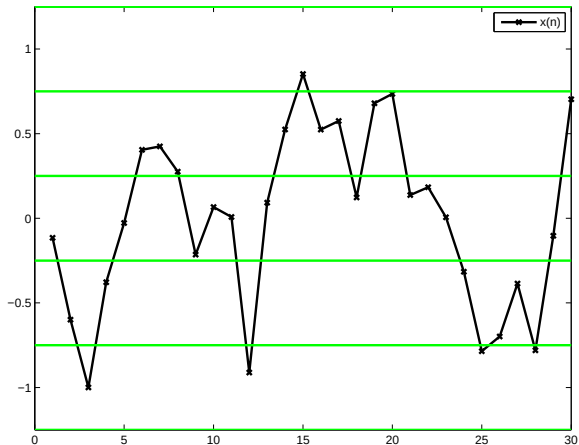


Example 1



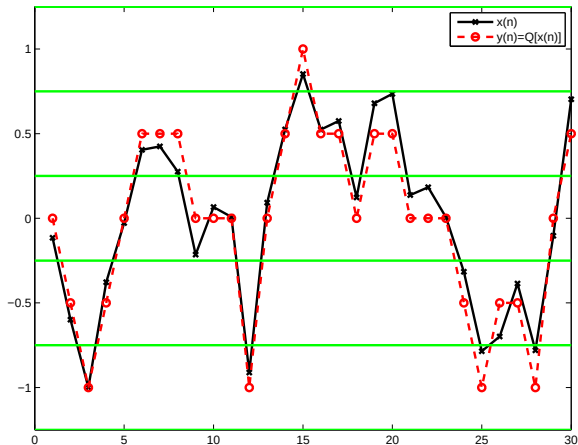


Example 1



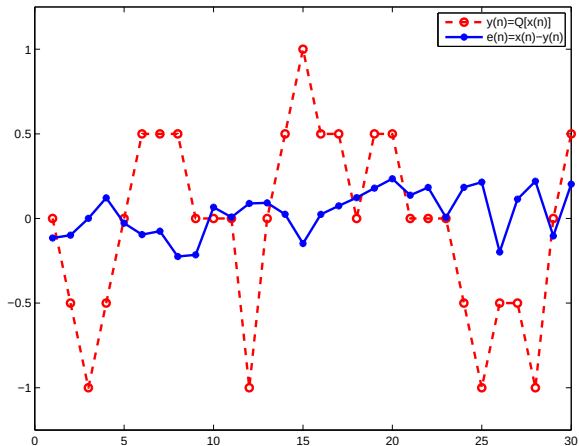


Example 1





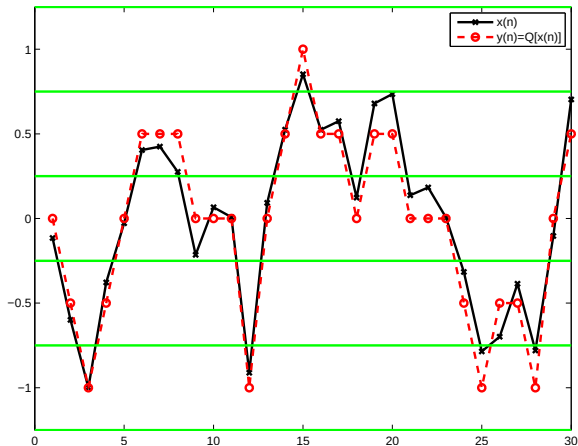
Example 1







Example 1



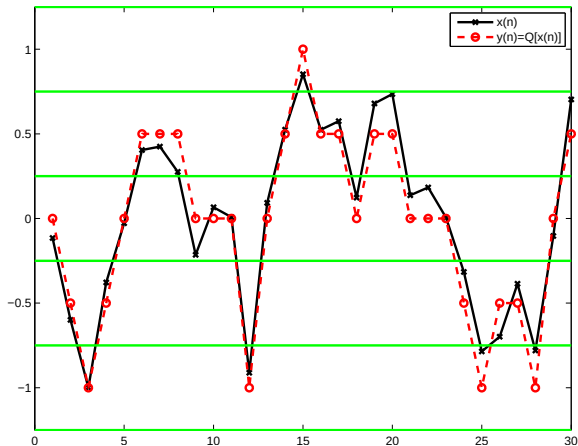
Go to wooclap.com with the code **PBTFOU**

Input interval: $(-A, A) = (-1.25, 1.25)$

Thresholds:



Example 1



Go to wooclap.com with the code **PBTFOU**

Input interval: $(-A, A) = (-1.25, 1.25)$

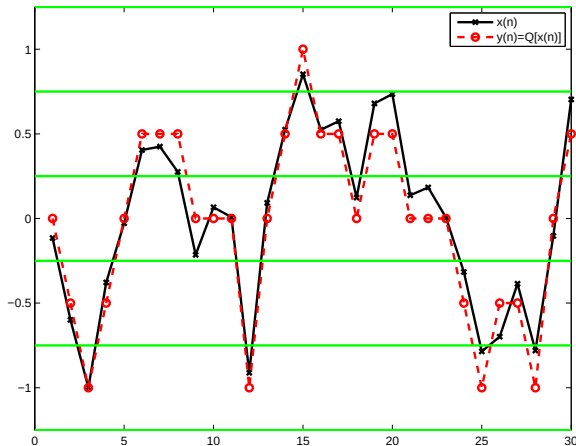
Thresholds: $-1.25, -0.75, -0.25, 0.25, 0.75, 1.25$







Example 1



Go to wooclap.com with the code **PBTFOU**

Input interval: $(-A, A) = (-1.25, 1.25)$

Thresholds: $-1.25, -0.75, -0.25, 0.25, 0.75, 1.25$

Levels: $-1, -0.5, 0, 0.5, 1$

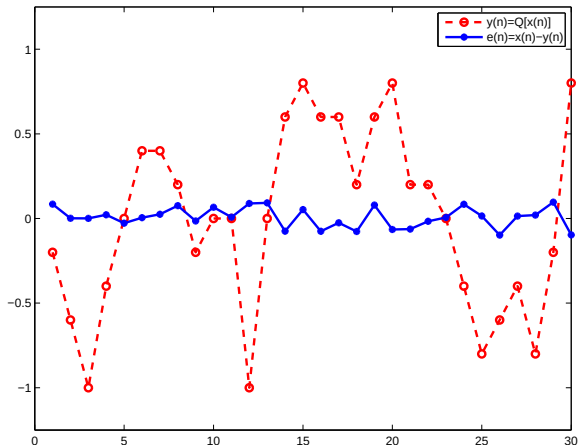
Note that:

- ▶ All the cells have the same size
 $\Delta = \frac{2A}{L} = 2.5/5 = 0.5$
- ▶ Each level is the center of the corresponding cell





Example 2



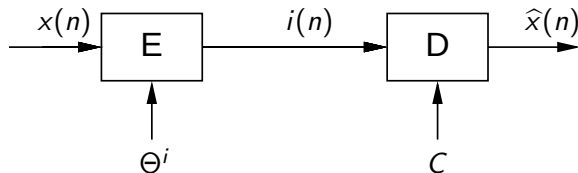
Here we have again cells with identical size and levels at the center of the cell

The quantization cells are smaller $\Delta = 0.2$

The quantization error is typically smaller



Quantization as coding / decoding



- ▶ The quantization process can be seen as the cascade of “encoding” and “decoding”
- ▶ The encoder produces the index of the quantization region, $i(n) \in \mathbb{Z}$
- ▶ The number $i(n)$ is represented with a suitable string of bits, e.g by using *lossless coding*
- ▶ The decoder associates to $i(n)$ to the corresponding level (or code-word) $\hat{x}^i(n)$
- ▶ In a compression system, often with the term quantization we just refer to the operation $x \rightarrow i$, while the evaluation of the quantization level, $i \rightarrow \hat{x}^i$ is referred to as *Inverse Quantization*
 - ▶ The first is performed at the encoder, the second at the decoder



Rate of a scalar quantizer

- ▶ **Rate** R and **distortion** D are the performance criteria of a quantizer
- ▶ The rate of a scalar quantizer is the average number of bits needed to represent a sample, or rather, its quantization index: it amounts thus to the binarization of $i(n)$
- ▶ It is *always done via lossless coding*
- ▶ We will assume $R = \log_2 L$
- ▶ This is a good approximation of real-life coding rate, with two implicit assumption



Rate of a scalar quantizer

- ▶ **Rate** R and **distortion** D are the performance criteria of a quantizer
- ▶ The rate of a scalar quantizer is the average number of bits needed to represent a sample, or rather, its quantization index: it amounts thus to the binarization of $i(n)$
- ▶ It is *always done via lossless coding*
- ▶ We will assume $R = \log_2 L$
- ▶ This is a good approximation of real-life coding rate, with two implicit assumption
 - ▶ Quantized data have a uniform probability distribution (pessimistic)
 - ▶ The binary representation of $i(n)$ is the best (entropy coder)
- ▶ We will elaborate on this problem in the next lesson



Distortion of a scalar quantizer

- We use the squared error, which, on a single sample is:

$$d[x(n), Q[x(n)]] = |e(n)|^2 = |x(n) - Q[x(n)]|^2$$

- For a signal $x(\cdot)$ of duration N , we use the mean square error (MSE):

$$D = \frac{1}{N} \sum_{n=0}^{N-1} d[x(n), Q[x(n)]]$$

- ▶ For random signals¹, $D = E \left\{ |X(n) - Q(X(n))|^2 \right\} = E \left\{ |E(n)|^2 \right\}$
- ▶ In this case, distortion is the variance of the random process $E(n) = X(n) - Q(X(n))$, and is indicated as σ_Q^2

¹Notice that we use lower-case letters for deterministic signals and upper-case letter for random variables/signals, while $\mathbb{E}$ is the expectation operator.



Mathematical reminders: integer mapping

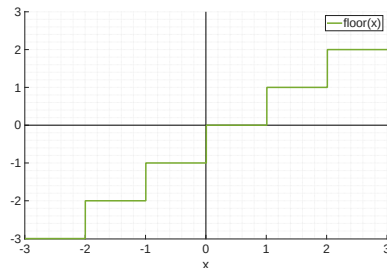
- ▶ Let us remind the integer mapping functions commonly used in programming:
floor, ceil, round, fix
- ▶ They are “staircase functions”
- ▶ They are widely used to implement quantizers



Mathematical Reminders: Floor Function

Operator: $|x|$

- ▶ **Definition:** The greatest integer less than or equal to x
- ▶ **Behavior:** Always rounds towards $-\infty$
- ▶ **Examples:**
 - ▶ $\lfloor 2.7 \rfloor = 2$
 - ▶ $\lfloor -2.7 \rfloor = -3$
- ▶ **Usage:** Common in basic uniform quantization models

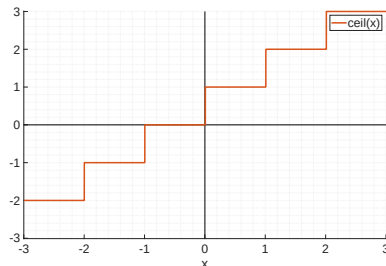




Mathematical Reminders: Ceil Function

Operator: $\lceil x \rceil$

- ▶ **Definition:** The smallest integer greater than or equal to x
- ▶ **Behavior:** Always rounds towards $+\infty$
- ▶ **Examples:**
 - ▶ $\lceil 2.1 \rceil = 3$
 - ▶ $\lceil -2.1 \rceil = -2$
- ▶ **Usage:** Used for determining buffer sizes or resource allocation bounds

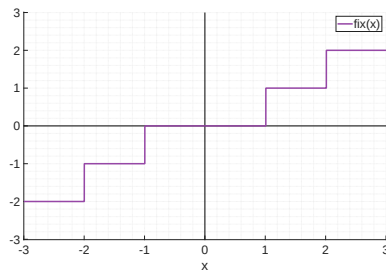




Mathematical Reminders: Fix Function

Operator: $\text{fix}(x)$

- ▶ **Definition:** Drops the fractional part of the number
- ▶ **Behavior:** Rounds towards zero (Symmetric)
 - ▶ Equivalent to $\lfloor x \rfloor$ for $x \geq 0$.
 - ▶ Equivalent to $\lceil x \rceil$ for $x < 0$.
- ▶ **Examples:**
 - ▶ $\text{fix}(2.7) = 2$
 - ▶ $\text{fix}(-2.7) = -2$
- ▶ **Note:** Standard behavior for integer casting in most programming languages

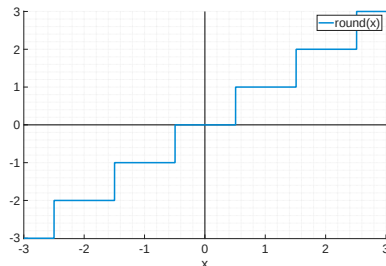




Mathematical Reminders: Round Function

Operator: $\text{round}(x)$

- ▶ **Definition:** Maps x to the nearest integer
- ▶ **Behavior:** Usually implemented as $\lfloor x + 0.5 \rfloor$
- ▶ **Examples:**
 - ▶ $\text{round}(2.51) = 3$
 - ▶ $\text{round}(2.49) = 2$
 - ▶ $\text{round}(-2.3) = -2$
- ▶ **Usage:** allow a straightforward implementation of *uniform quantization*





Outline

Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization



Uniform quantization

Uniform quantization is characterized by:

- ▶ The input range: $(0, A)$ (unsigned data) or $(-\frac{A}{2}, \frac{A}{2})$ (signed data)
- ▶ The number of levels L

The input range is divided into L equal-sized cells. The cell size is $\Delta = \frac{A}{L}$. Each cell is represented by its mid-point, i.e. the quantization levels are the centers of the quantization cells.

$$\forall i, \Delta^i = \Delta = A/L$$

$$t^i = t^{i-1} + \Delta \text{ (red bars)}$$

$$\hat{x}^i = \frac{t^i + t^{i-1}}{2} \text{ (green crosses)}$$

$$\Theta^i = \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2} \right)$$





Uniform Quantization for unsigned data

For **unsigned data** (range $(0, A)$) the thresholds are $(0, \Delta, 2\Delta, \dots, L\Delta)$, i.e., $t^i = (i-1)\Delta$.

We can thus define the behavior of the **encoder** (from x to the quantization index i), of the **decoder** (from i to the quantization level $\hat{x}^i$) and of the full quantization chain:

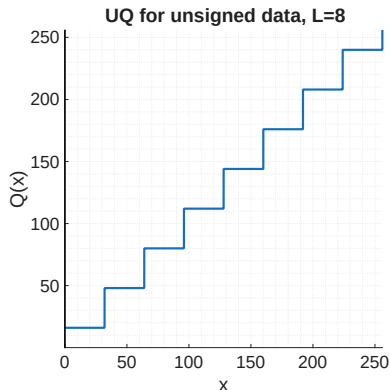
$$i = \left\lceil \frac{x}{\Delta} \right\rceil$$

encoder

$$\hat{x}^i = i \cdot \Delta - \frac{\Delta}{2}$$

decoder

$$Q(x) = \Delta \cdot \left\lceil \frac{x}{\Delta} \right\rceil - \frac{\Delta}{2}$$





Uniform quantization for signed data

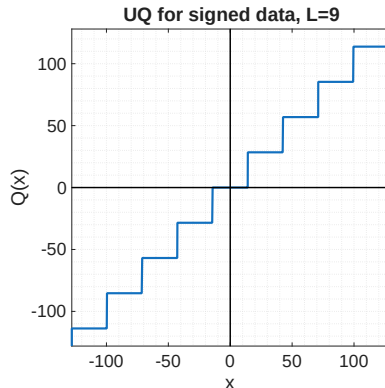
For **signed data** (range in $(-A/2, A/2)$) one typically wants zero to be a quantization level (**mid-tread** quantizer): in such a way, small fluctuations are quantized to zero. Therefore, we **always use an odd number of levels for UQ of signed data**. In this case, uniform quantization (UQ) amounts to **rounding**:

$$i = \text{round} \left(\frac{x}{\Lambda} \right) \quad \text{encoder}$$

$$\hat{x}^i = \Delta \cdot i \quad \text{decoder}$$

$$Q(x) = \Delta \cdot \text{round} \left(\frac{x}{\Delta} \right)$$

In other words, $Q(x)$ is the closest multiple of Δ . This quantizer is also referred to as **mid-tread** quantizer.



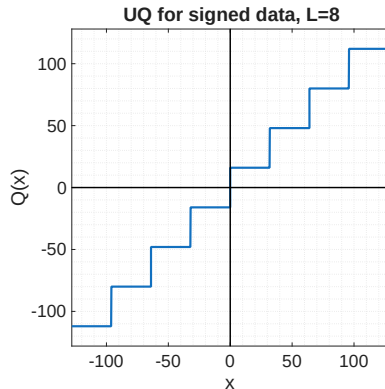


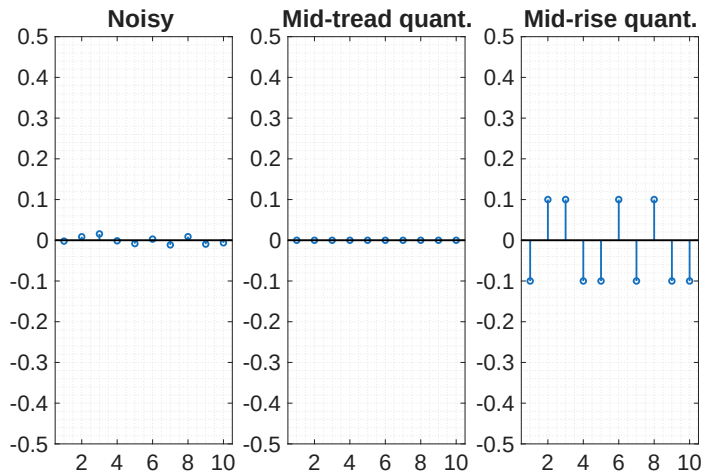
Uniform quantization for signed data

If one uses uniform quantization with an even number of levels for signed data, the central threshold $t^{\frac{N}{2}}$ is zero, see figure on the right.

This quantizer is also referred to as **mid-rise** quantizer. Since zero is a threshold any very small value will be quantized as $\pm\Delta/2$ according to its sign.

This can be a problem if the signal has small fluctuations around zero due to additive noise: quantization enhances noise!





On the left, we show $x(n)$, a signal with small values that are **interpreted** as noise: uninformative, random fluctuations.

In the middle, we show $Q(x)$ when zero is a quantization level (**mid-tread** quantizer): the small values are forced to zero, making it “easier” to encode the signal.

On the right, we show $Q(x)$ when zero is a threshold (**mid-rise** quantizer): after quantization the noise is amplified instead of being removed!

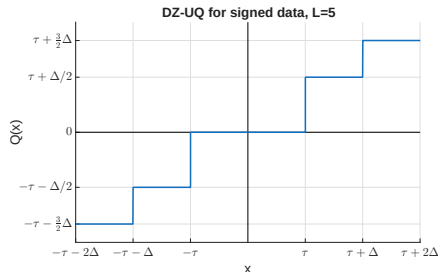


Deadzone quantization

Pushing further this idea, we can design a quantizer where the central cell is larger than the others. This allows for removing more small-valued oscillations, while keeping a fine resolution on non-zero values. This is called **deadzone** quantization (DZQ) and is very commonly used in data compression. Using the threshold value τ , the DZQ can be implemented as:

$$i = \begin{cases} \text{sign}(x) \cdot \left\lfloor \frac{|x| + \frac{\tau\Delta}{2}}{\Delta} \right\rfloor & \text{if } |x| \geq \tau \\ 0 & \text{if } |x| < \tau \end{cases} \quad \text{encoder}$$

$$\hat{x} = \begin{cases} \text{sign}(i) \cdot \Delta \left(|i| + \frac{1-\tau}{2} \right) & \text{if } i \neq 0 \\ 0 & \text{if } i = 0 \end{cases} \quad \text{decoder}$$





UQ: wrap-up

- ▶ UQ is intuitive: just split the input range into L intervals, and represent each interval with its middle.
- ▶ Implementation is easy:

$$Q(x) = \Delta \cdot \text{round} \left(\frac{x}{\Delta} \right) \quad \text{mid-tread}$$

$$Q(x) = \Delta \cdot \text{floor} \left(\frac{x}{\Delta} \right) + \frac{\Delta}{2} \quad \text{mid-rise}$$

- ▶ For a given range A and a given number of levels L , UQ has the smallest maximum error (optimal **minimax** quantizer)
 - ▶ The maximum error is half the size of the maximum quantization interval: any modification of the UQ increases the maximum quantization interval and thus the maximum error
- ▶ Last, but not least, UQ allows for an analytical formula of the RD (rate-distortion) curve. But before that...





Image quantization: example 1







A close-up photograph of a variety of colorful peppers. In the center, a large green bell pepper is prominent. To its left is a long, slender green chili pepper. Above the green bell pepper is a large, vibrant red bell pepper. To the right of the green bell pepper is a yellow bell pepper. The background is filled with more peppers in various colors and shapes, including smaller red and green peppers. The lighting is bright, highlighting the glossy texture of the peppers.



A close-up photograph of a variety of colorful bell peppers. In the center, a large green bell pepper is prominent. To its left is a long, slender yellow-green pepper. Above the green pepper is a large, vibrant red bell pepper. To the right of the green pepper is a yellow bell pepper. The background is filled with more peppers in various colors, including red, green, and yellow, creating a rich, textured scene.



A close-up photograph of a variety of colorful peppers. In the center, a large, vibrant red bell pepper is prominent. To its left, a long, slender, light green chili pepper stands vertically. In the foreground, a large, green bell pepper is visible. The background is filled with other peppers, including smaller green and yellow ones, creating a rich, textured composition. The lighting is bright, highlighting the glossy surfaces of the peppers.



A close-up photograph of various colorful peppers. In the center, a large, glossy red bell pepper is prominent. To its left, a long, slender yellow chili pepper stands vertically. In the foreground, a large green bell pepper is visible. The background is filled with other peppers in shades of red, yellow, and green, creating a vibrant and textured composition.



Image quantization: example 1

Rate 6 bpp	PSNR 27.83 dB	Compression ratio 4.000
------------	---------------	-------------------------





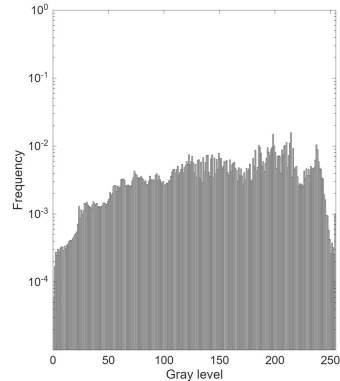
Image quantization: example 1

Rate 3 bpp	PSNR 25.75 dB	Compression ratio 8.000
------------	---------------	-------------------------



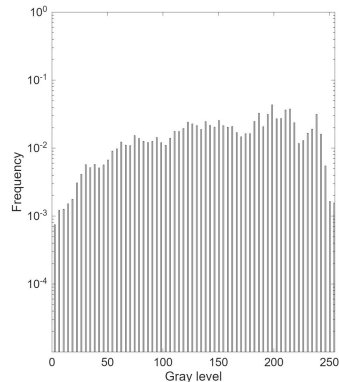


Scalar quantization: example 2



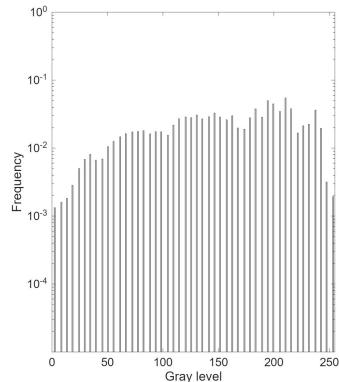


Scalar quantization: example 2



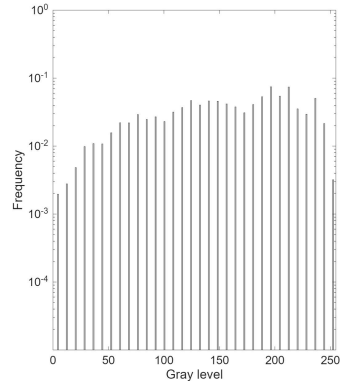


Scalar quantization: example 2



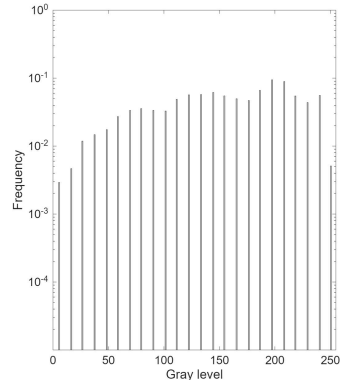


Scalar quantization: example 2



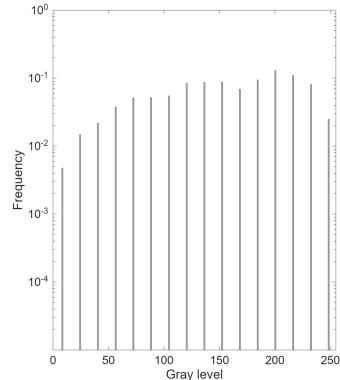


Scalar quantization: example 2



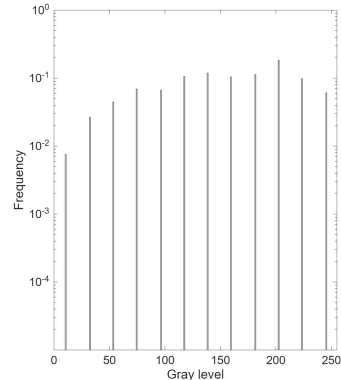


Scalar quantization: example 2



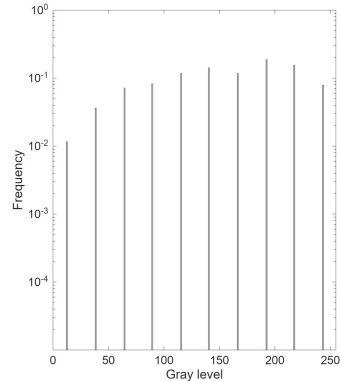


Scalar quantization: example 2



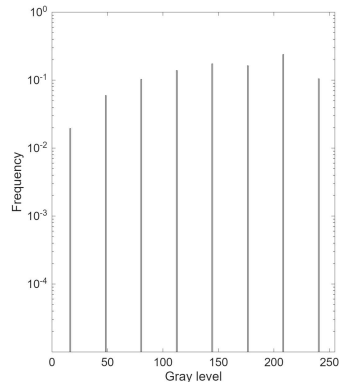


Scalar quantization: example 2



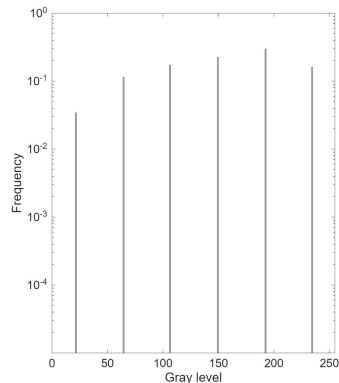


Scalar quantization: example 2



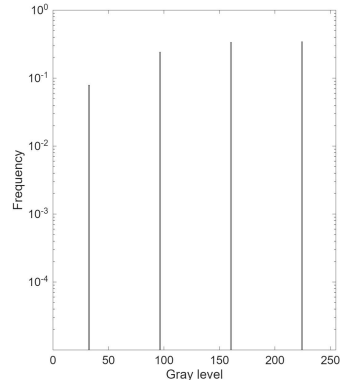


Scalar quantization: example 2



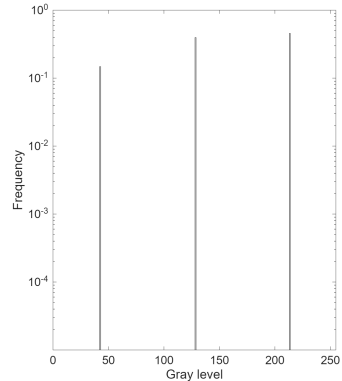


Scalar quantization: example 2



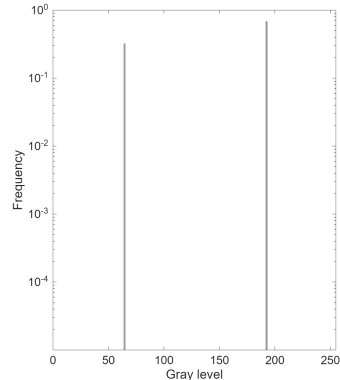
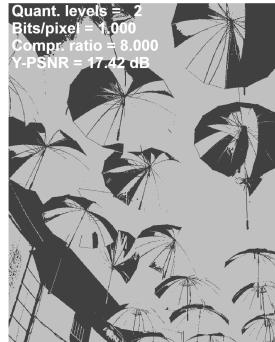


Scalar quantization: example 2



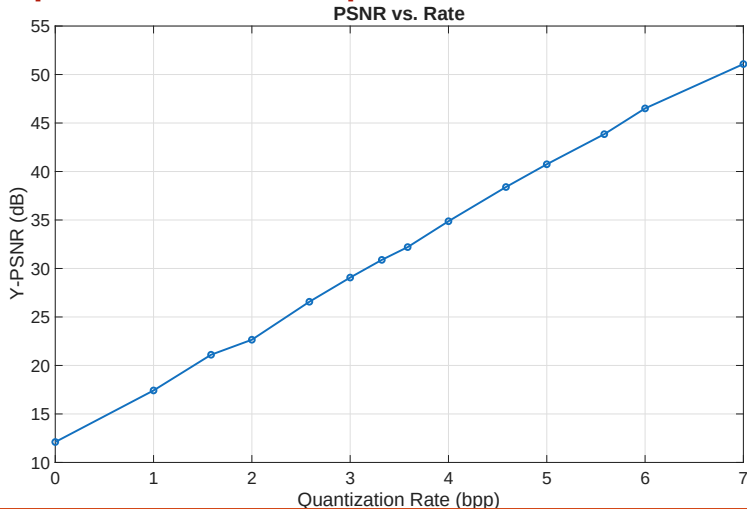


Scalar quantization: example 2





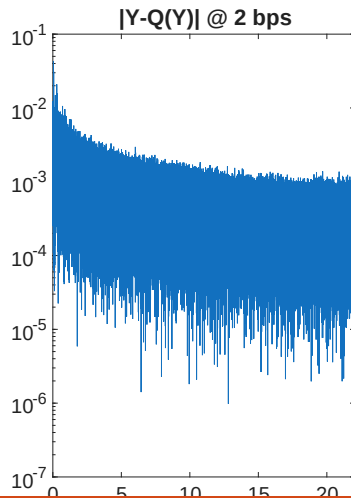
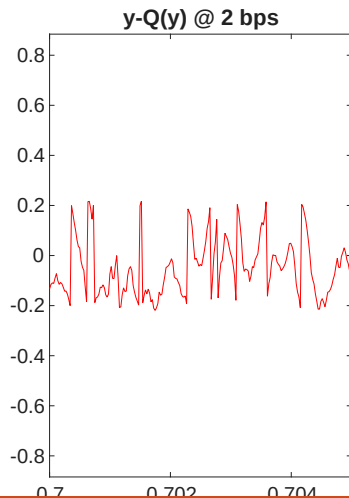
Scalar quantization: example 2







Audio quantization example







Outline

Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization

Probability Theory Refreshers

1. Uniform Random Variable



Probability Theory Refreshers

1. Uniform Random Variable

- ▶ A continuous random variable X is said to be **uniformly distributed** on the interval $[a, b]$ if its Probability Density Function (PDF) is defined as:

$$X \sim \mathcal{U}(a, b) \Leftrightarrow f_X(x) = \begin{cases} \frac{1}{b-a} & \text{for } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

- ▶ Its variance is $\text{Var}(X) = \frac{(b-a)^2}{12}$

2. Expected Value of a Function of an R.V. (LOTUS)



Uniform Quantization: RD curve

Let us apply these results to find the distortion of a uniform RV quantized with UQ.

Hypotheses : $X \sim \mathcal{U}(-\frac{A}{2}, \frac{A}{2})$ and $Q(x)$ is a uniform quantizer with L levels.

Goal: Find $\sigma_Q^2 = \mathbb{E}[(X - \hat{X})^2]$ as a function of A, L



Uniform Quantization: RD curve

Let us apply these results to find the distortion of a uniform RV quantized with UQ.

Hypotheses : $X \sim \mathcal{U}(-\frac{A}{2}, \frac{A}{2})$ and $Q(x)$ is a uniform quantizer with L levels.

Goal: Find $\sigma_Q^2 = \mathbb{E}[(X - \hat{X})^2]$ as a function of A, L

We observe that $\sigma_Q^2 = \mathbb{E}[g(X)]$, where $\forall u \in \mathbb{R}, g(u) = (u - Q(u))^2$.

$$\sigma_Q^2 = \mathbb{E}[(X - \hat{X})^2] = \int_{-\infty}^{+\infty} p_X(u)[u - Q(u)]^2 du$$





Uniform Quantization: RD curve

We can use the previous result to determine the **rate-distortion** curve of the UQ, i.e. the mathematical relationship between the rate R and the distortion $D = \sigma_Q^2$.

$$D = \frac{\Delta^2}{12} = \frac{A^2}{12L^2} = \frac{A^2/12}{\cdot 2^{2R}} = \sigma_X^2 2^{-2R}$$

$$\begin{aligned} \text{SNR} &= 10 \log_{10} \frac{E\{X^2\}}{D} = 10 \log_{10} \frac{\sigma_X^2}{\sigma_X^2 2^{-2R}} \\ &= 10 \log_{10} 2^{2R} \approx 6R \end{aligned}$$



High Resolution (HR) Uniform Quantization

- ▶ Hypothesis : $L \rightarrow +\infty$, X generic RV
- ▶ In HR, in any given Θ^i we approximate p_X as a constant (it is constant in the region, but the value can be different in another region)
- ▶ Therefore, the quantization noise in Θ^i is $\mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2})$
- ▶ From the total probability law, $E \sim \mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2})$
- ▶ Thus:

$$D = \frac{\Delta^2}{12} = \frac{A^2}{12} 2^{-2R}$$





Outline

Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization



Optimal quantization

For a given probability density function $p_X(x)$, we want to find the quantizer minimizing distortion for a given rate

Problem equivalent to determine thresholds t^i and levels $\hat{x}^i$.

Solutions :

- ▶ Analytic Solution in *high resolution*: $D = c_X \sigma_X^2 2^{-2R}$
- ▶ If the high resolution hypothesis is not met, we can find a local optimum with the Lloyd-Max algorithm



Optimal quantization in high resolution

- ▶ The optimal quantizer is the one that, for a given PDF p_X of the input samples, provides the least distortion $D = \mathbb{E}[(X - Q(X))^2]$ for a given rate R
- ▶ It is possible to find the *rate-distortion* curve for the optimal quantizer in the **high-resolution hypothesis**:
 - ▶ $L \rightarrow \infty$
 - ▶ $\max_i \Delta_i \rightarrow 0$
 - ▶ $\forall i, \forall u \in \Theta^i, p_X(u) \approx P_i$
- ▶ Then it can be shown that the optimal quantizer has the following RD curve:

$$\sigma_Q^2 = c_X \sigma_X^2 2^{-2R} \quad \text{with} \quad c_X = \frac{1}{12} \left[\int_{\mathbb{R}} p_U^{1/3}(t) dt \right]^3 \quad \text{and} \quad U = \frac{X}{\sigma_X}$$





Non-HR Optimal quantization

- ▶ High resolution hypothesis is not always met
- ▶ On the contrary, often quantization must work at low or very low rate
- ▶ **There is no analytical solution for low resolution optimal quantization**
- ▶ On the other hand, it is possible to find **necessary conditions** for optimal quantizers independently from the resolution
- ▶ These conditions allows to define an algorithm attaining a local optimum point (Lloyd-Max algorithm)



Optimal quantization

Lloyd-Max algorithm

1. Choose an initial dictionary $\mathcal{C}^{(0)} = \{\hat{x}_0^i\}_{i=1,\dots,L}$
2. Find the *best* thresholds for the given dictionary
3. Find the *best* dictionary for the given thresholds
4. Iterate 2-3 until a stop condition is met



Lloyd-Max algorithm: initial dictionary

- ▶ Uniform Initialization: $\hat{x}^n = -A_0 + n\Delta \forall n = 1, \dots, L$
- ▶ Random Initialization: code-words are initialized with the same distribution as data



Lloyd-Max algorithm: nearest neighbor rule

- ▶ Given $\{\hat{x}^i\}_{i=1,\dots,L}$, we have to define, for each input, which is the best level to use as its representative
- ▶ The best choice is obviously the nearest neighbor:

$$k = \arg \min_n |x - \hat{x}^n| \Rightarrow Q(x) = \hat{x}^k$$

- ▶ Choosing the next neighbor is equivalent to set the threshold at the midpoint between consecutive quantization levels:

$$t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2}, \quad i \in \{1, \dots, L-1\}$$



Lloyd-Max algorithm: centroid condition

Given the thresholds $\{t^0, \dots, t^L\}$, the best levels are those minimizing the distortion:

$$\sigma_Q^2 = \sum_{i=1}^L \int_{t^{i-1}}^{t^i} (u - \hat{x}^i)^2 p_X(u) du$$

To find the optimal levels, we compute the gradient of the distortion and set it to 0.

$$\begin{aligned} \frac{\partial \sigma_Q^2}{\partial \hat{x}^i} &= \frac{\partial}{\partial \hat{x}^i} \int_{t^{i-1}}^{t^i} (u - \hat{x}^i)^2 p_X(u) du = \int_{t^{i-1}}^{t^i} \frac{\partial}{\partial \hat{x}^i} (u - \hat{x}^i)^2 p_X(u) du \\ &= \int_{t^{i-1}}^{t^i} 2(\hat{x}^i - u) p_X(u) du = 2\hat{x}^i \int_{t^{i-1}}^{t^i} p_X(u) du - 2 \int_{t^{i-1}}^{t^i} u p_X(u) du \\ \hat{x}^i &= \frac{\int_{t^{i-1}}^{t^i} u p_X(u) du}{\int_{t^{i-1}}^{t^i} p_X(u) du} = \frac{\int_{\Theta_i} u p_X(u) du}{\int_{\Theta_i} p_X(u) du} = \mathbb{E}[X | X \in \Theta_i] \end{aligned}$$

This is the so-called centroid condition: the best representative of a quantization region is its centroid, i.e. the conditional average of X given $X \in \Theta^i$



Lloyd-Max algorithm: stop condition

- ▶ After the k -iteration of NN and the centroid rules, we can compute the new distortion value $\sigma_{Q,(k)}^2$
- ▶ It can be compared with the previous distortion $\sigma_{Q,(k-1)}^2$: the optimization steps assure that the new distortion can only be less than or equal to the old one
- ▶ We compute the relative distortion reduction: if it is smaller than a threshold, we stop the algorithm:

$$\frac{\sigma_{Q,(k-1)}^2 - \sigma_{Q,(k)}^2}{\sigma_{Q,(k-1)}^2} < \varepsilon$$

- ▶ Another stop condition is upon a given number of iterations:

$$k = K$$



Convergence of Lloyd-Max algorithm

- ▶ Convergence to global optimum is not assured
- ▶ Final result depends on **initialization**
- ▶ However, at each step distortion cannot increase



Lloyd-Max algorithm for data

- ▶ In real-life problems, we have not signal PDFs, but rather a bunch of data
- ▶ We can modify Lloyd-Max algorithm as follows:
 1. Let $\mathcal{X} = \{u_1, u_2, \dots, u_M\}$ be the set of data to quantize (or the training set if data is not known in advance)
 2. Initialize ($k=0$) with any dictionary (e.g. uniform): $\mathcal{C}^{(k)} = \{\hat{x}_0^i\}_{i=1, \dots, L}$
 3. Nearest neighbor rule:

$$W_k^i = \{u_m \in \mathcal{X} : \forall j \neq i \|u_m - \hat{x}_k^i\| \leq \|u_m - \hat{x}_k^j\|\}$$

4. Centroid rule: $\hat{x}_{k+1}^i = \frac{1}{|W_k^i|} \sum_{u_m \in W_k^i} u_m$
5. Iterate to step 3 until convergence



Scalar quantization: summary

- ▶ $D(R)$, UQ of uniform RV: $D = \sigma_X^2 2^{-2R}$
- ▶ $D(R)$, UQ of non uniform RV in HR: $D = K_X \sigma_X^2 2^{-2R}$
- ▶ $D(R)$, OQ in HR (any RV): $\sigma_Q^2 = c_X \sigma_X^2 2^{-2R}$
- ▶ c_X is the shape factor
 - ▶ $c_X = 1$ for uniform RVs
 - ▶ $c_X = \frac{\sqrt{3}}{2} \pi$ for Gaussian RVs
- ▶ At any resolution, the LMA produces a non-uniform quantizer which is locally optimal



Definitions

Uniform Quantization

Examples

Rate-distortion curve

Optimal Scalar Quantization

Predictive scalar quantization

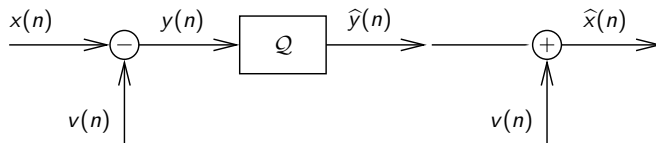




Coding scheme

Open loop scheme

- ▶ $x(n)$ is predicted from the past
- ▶ If the prediction is good, $v(n) \approx x(n)$



- ▶ How to provide $v(n)$?
- ▶ What is the gain?



Prediction gain

Prediction Error = Signal Error :

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - \hat{x}(n) + v(n) = \bar{q}(n)$$

Thus the goal of predictive quantization (PQ) is to minimize the distortion of y
The coding gain is:

$$\text{SNR}_p = 10 \log_{10} \frac{\sigma_X^2}{D} = 10 \log_{10} \frac{\sigma_X^2}{\sigma_Y^2} + 10 \log_{10} \frac{\sigma_Y^2}{D} = G_P + G_Q$$

Prediction is effective if and only if the prediction error has a smaller variance than the original signal



Go to wooclap.com with the code **PBTFUO**

$$X(n) \sim \mathcal{N}(0, \sigma^2)$$

$$V(n) = X(n - 1)$$

$$\mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{|n-m|}$$

$\rho : G_P > 0 ?$



Predictors

- ▶ Linear Predictors are simple and moreover optimal for Gaussian RV

$$v(n) = - \sum_{i=1}^P a_i x_{n-i} \quad \text{Filter with } P \text{ parameters}$$

$$y(n) = x(n) - v(n) = \sum_{i=0}^P a_i x_{n-i} \quad \text{Prediction error}$$

- ▶ with $a_0 = 1$.
- ▶ y is the result of filtering x with

$$A(z) = 1 + a_1 z^{-1} + \dots + a_P z^{-P}$$

- ▶ Optimal filter: minimization of σ_Y^2



Predictor selection

Problem:

Find the vector $\underline{a}$ minimizing

$$\sigma_Y^2 = \mathbb{E} \{ Y^2(n) \} = \mathbb{E} \left\{ \left[X(n) + \sum_{i=1}^P a_i X(n-i) \right]^2 \right\}$$



Predictor selection

$$\begin{aligned}\sigma_Y^2 &= E\{X^2(n)\} + 2 \sum_{i=1}^P a_i E\{X(n)X(n-i)\} + \sum_{i=1}^P \sum_{j=1}^P a_i a_j E\{X(n-i)X(n-j)\} \\ &= \sigma_X^2 + 2\mathbf{r}^t \mathbf{a} + \mathbf{a}^t \mathbf{R}_X \mathbf{a}\end{aligned}$$

with:

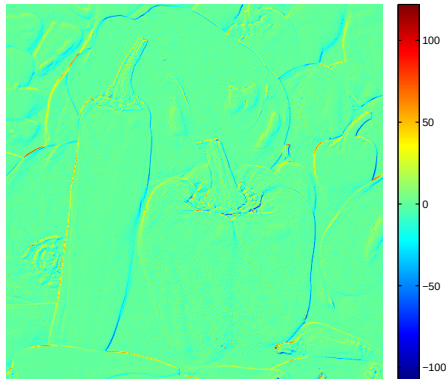
$$\mathbf{r} = [r_X(1) \dots r_X(P)] \quad \mathbf{R}_X = \begin{bmatrix} r_X(0) & r_X(1) & r_X(2) & \dots & r_X(P-1) \\ r_X(1) & r_X(0) & r_X(1) & \dots & r_X(P-2) \\ r_X(2) & r_X(1) & r_X(0) & \dots & r_X(P-3) \\ \dots & \dots & \dots & \dots & \dots \\ r_X(P-2) & r_X(P-3) & r_X(P-4) & \dots & r_X(1) \\ r_X(P-1) & r_X(P-2) & r_X(P-3) & \dots & r_X(0) \end{bmatrix}$$

$$r_X(k) = E\{X(n)X(n-k)\}$$





Predictive quantization: example



For the “peppers” image (grayscale), we compute the prediction error using a second order linear predictor:

$$\hat{f}_{n,m} = af_{n-1,m} + bf_{n,m-1}$$

The predictor of $f_{n,m}$ is a weighted average of the top and the left neighbors.

a	b	σ_Y^2
0	0	2902.7
1/2	1/2	78.7
0.449	0.546	78.4

The first row correspond to no prediction: thus $\sigma_X^2 = 2902.7$

The second row show the distortion of the simplest predictor: the average of the two neighbors. The distortion is reduced by a factor greater than 37

Using the optimal predictor, computed with the formulas shown in the previous slide, only slightly improves the performance



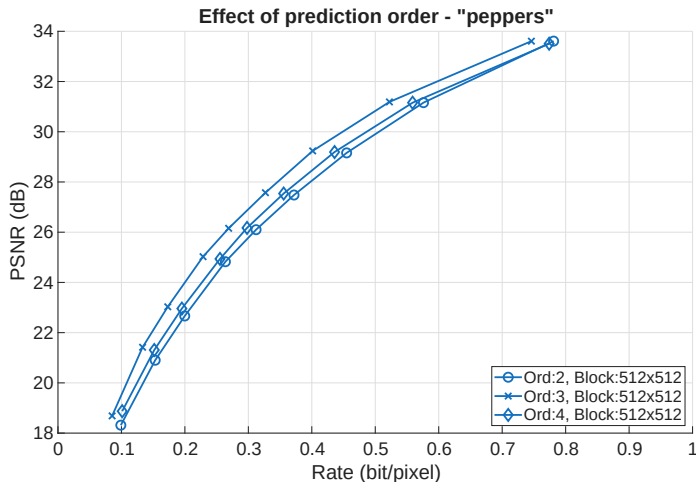
Impact of Filter Order on Prediction Gain

In global DPCM, the predictor $v(n)$ uses a linear combination of N causal neighbors.

- ▶ **Order Selection:** Increasing the order from 2 to 4 typically improves the **Prediction Gain** ($G_p = \sigma_X^2 / \sigma_Y^2$), leading to a more "whitened" residual.
- ▶ **Screening Effect:** The benefit of including "distant" pixels diminishes rapidly. In typical image statistics, the immediate neighbors capture most of the mutual information. Distant pixels are often redundant as their correlation with the target is already "screened" by the closer neighbors.
- ▶ **Complexity vs. Performance:** Higher orders increase computational overhead with negligible PSNR gains for most natural textures.



Rate-Distortion Analysis: Filter Order

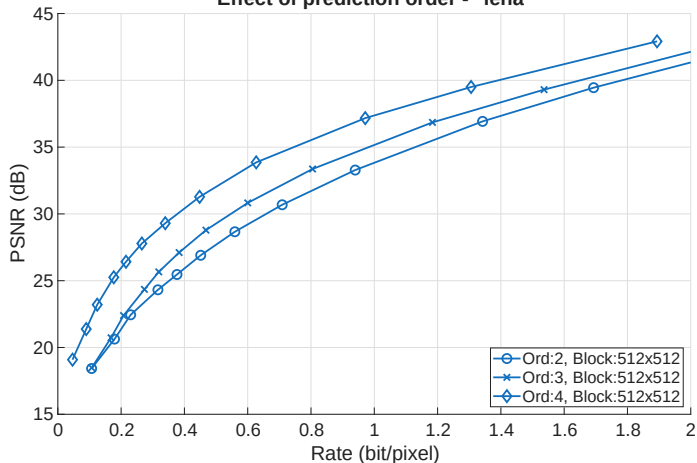


Homogeneous image with some background noise



Rate-Distortion Analysis: Filter Order

Effect of prediction order - "lena"



Clean image with different areas



Local Adaptation: The Block-wise Approach

Images are **non-stationary** signals; a single global filter is suboptimal for varying textures (e.g., edges vs. smooth areas). **The Strategy:** Divide the image into $M \times M$ blocks and

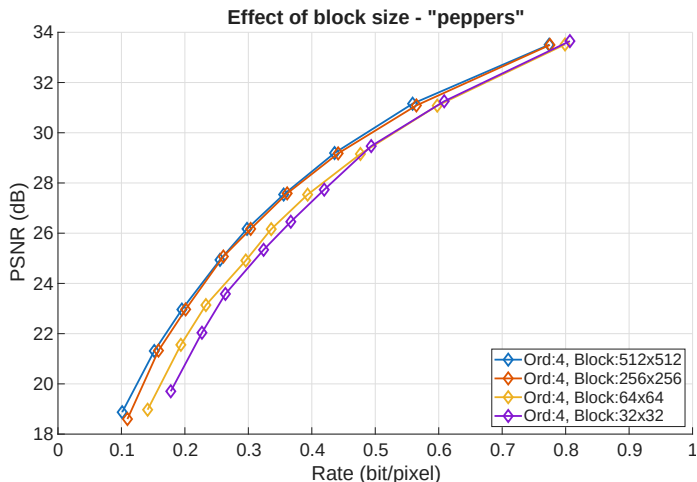
compute the **locally optimal** filter for each.

The Side Information Trade-off Total Rate (R_{total}) is now increased by $\frac{N \cdot B}{M^2}$ Where N is the filter order and B is the bits per coefficient (e.g., 16).

- ▶ **Small Blocks:** Excellent tracking of local statistics, but massive **overhead** from filter coefficients.
- ▶ **Large Blocks:** Minimal overhead, but the predictor fails to adapt to local features.



Rate-Distortion Analysis: Block size

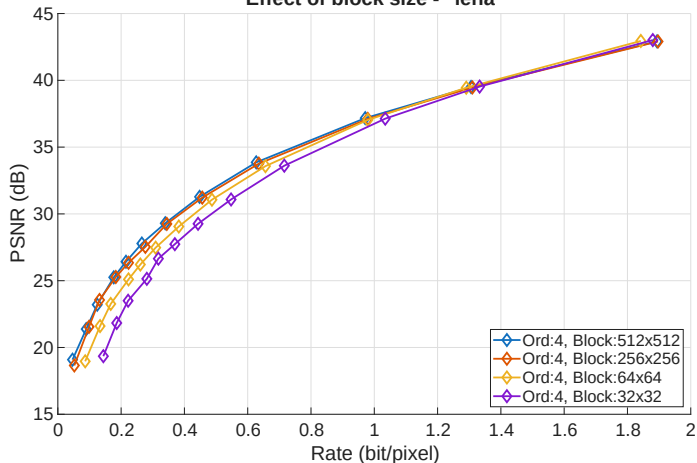


Homogeneous image with some background noise



Rate-Distortion Analysis: Block size

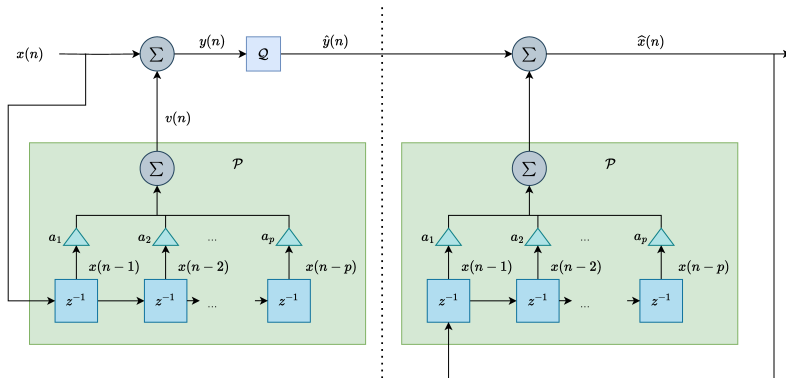
Effect of block size - "lena"



Clean image with different areas

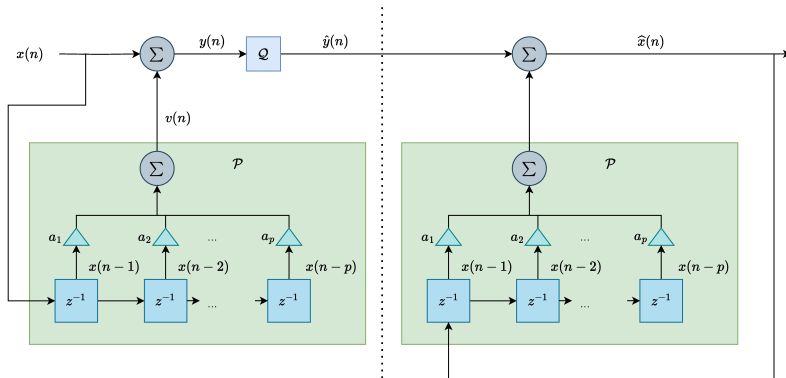


Coding/decoding scheme





Coding/decoding scheme



WRONG! The predictor operator is the same at the encoder and decoder but the input is different: $x \neq \hat{x}$! This causes **drift**.



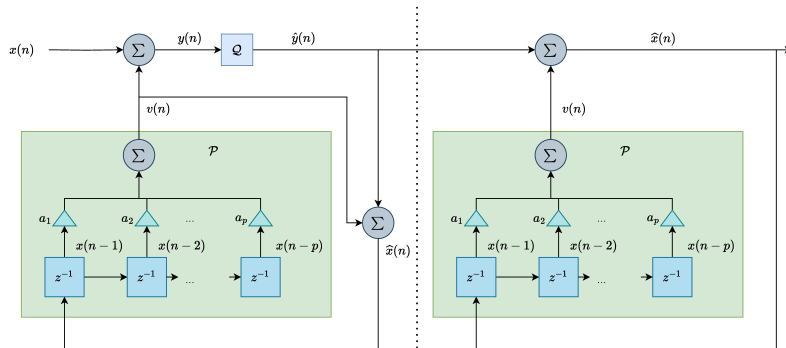
Predictive Quantization Drift Example

- ▶ Simple predictive scheme: prediction is the previous value
- ▶ 3-bit quantizer (7 levels): $[-9, -6, -3, 0, 3, 6, 9]$
- ▶ Encoder uses non-quantized values for prediction
- ▶ Decoder uses quantized (reconstructed) values for prediction

Step	Original Signal	Encoder			Decoder		
		Prediction	Error	Quantized Error	Prediction	Received Error	Reconstructed Value
1	10	-	-	9	-	9	9
2	11	10	1	0	9	0	9
3	12	11	1	0	9	0	9
4	13	12	1	0	9	0	9
5	14	13	1	0	9	0	9
6	18	14	4	3	9	3	12
7	21	18	3	3	12	3	15
8	18	21	-3	-3	15	-3	12



Coding/decoding scheme



This is the correct scheme: the predictor is fed with the same data (quantized past samples) both at the encoder and the decoder.



Predictive Quantization Drift Example

- ▶ Simple predictive scheme: prediction is the previous value
- ▶ 3-bit quantizer (7 levels): $[-9, -6, -3, 0, 3, 6, 9]$
- ▶ Encoder uses quantized values for prediction
- ▶ Decoder uses quantized (reconstructed) values for prediction

Step	Original Signal	Encoder			Decoder		
		Prediction	Error	Quantized Error	Prediction	Received Error	Reconstructed Value
1	10	-	-	9	-	9	9
2	11	9	2	3	9	3	12
3	12	12	0	0	12	0	12
4	13	12	1	0	12	0	12
5	14	12	2	3	12	3	15
6	18	15	3	3	15	3	18
7	21	18	3	3	18	3	21
8	18	21	-3	-3	21	-3	18

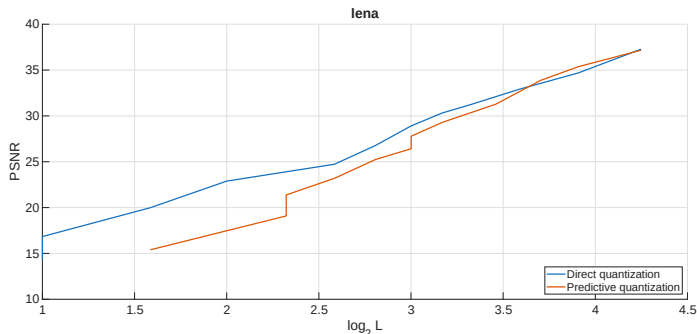


Coding scheme

- ▶ The encoder computes the prediction error $Y = X - V$
- ▶ The predictor V must be computed from data available at the decoder as well
- ▶ Thus the encoder must compute the reconstructed value $\hat{X}$
- ▶ This value is sent into a buffer and used to compute the prediction of future values



Direct vs. Predictive quantization

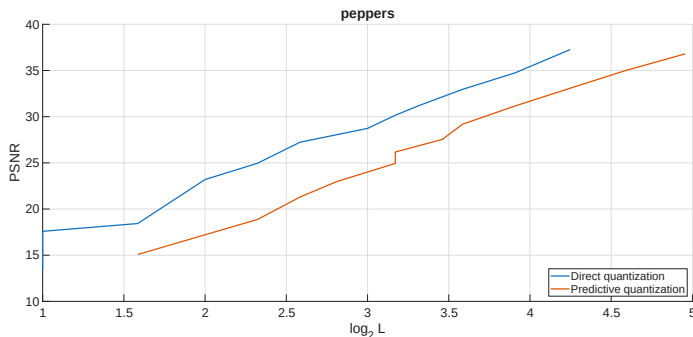


We show here the PSNR of the quantized image vs. the $\log_2 L$, which is our first rate estimation. The result seems very deceiving: apart from a short range of high rates, predictive quantization does not bring the expected gain: how is this possible?

The main explanation is that a practical PQ scheme must use the quantized data to produce a predictor, which is not what we considered in our theoretical analysis.



Direct vs. Predictive quantization



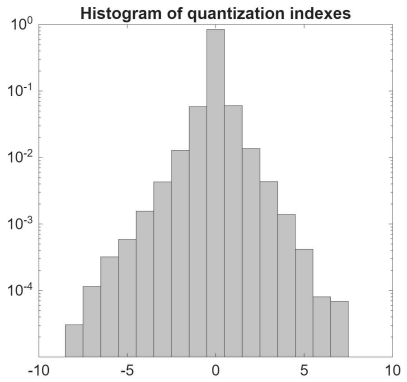
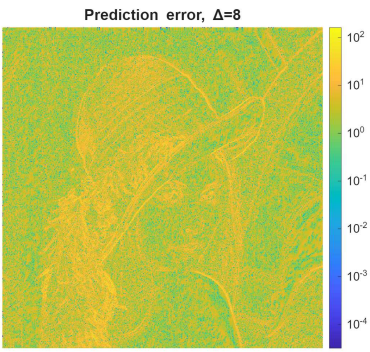
For the **peppers** image things are even worse.

- ▶ At low bitrates, the predictor is computed on heavily quantized data
- ▶ At high bitrates, the quantizer is obliged to reconstruct the additive noise affecting the image

However, the main point is that **the estimation of the rate with the logarithm is too much pessimistic**



PQ prediction error





PQ prediction error

Let us look more closely to the quantized prediction error. In this example, $L = 19$, so the estimated rate is $\log_2 19 \approx 4.25$ bits per sample.

However, we observe that most of the quantization indexes ($\approx 84\%$) are zero. We could use a **variable lenght code** that uses a short codeword for the index 0 and possibly longer codewords for indexes outside $(-3, 3)$, which account for less than 1% of the total.

Encoding the quantizations indexes with non-trivial encoding strategy (called **entropy coding**, as we will see later on) changes dramatically the results



Quantization: wrap-up

- ▶ UQ: $D = K_X \sigma_X^2 2^{-2R}$ for uniform RV or in high res
- ▶ Optimal SQ: similar behavior
- ▶ Both UQ and optimal SQ alone are catastrophic if applied directly on images or sound: the quality drops and the rate is not reduced much
- ▶ **Predictive quantization** can improve dramatically performance but:
 1. Never forget to compute the prediction on **quantized data**, otherwise the decoder will drift
 2. The input signal **must be correlated enough**
 3. **We must use the best encoding strategy to achieve good performance**

The best encoding strategy is **entropy coding**, the subject of the next lesson